

Deadlock

Antonio Pecchia

antonio.pecchia@unisannio.it

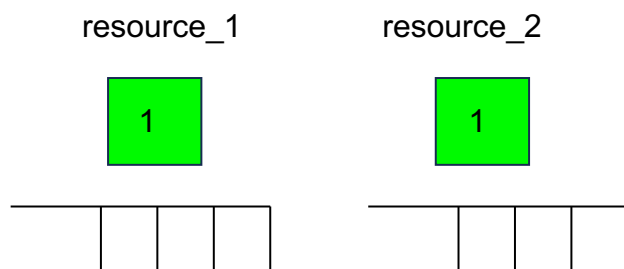
Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Esempio motivazionale

```
semaphore resource_1;  
semaphore resource_2;  
  
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}  
  
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```



Esempio motivazionale

```
semaphore resource_1;  
semaphore resource_2;
```

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

resource_1

0



resource_2

0



Esempio motivazionale

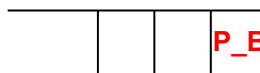
```
semaphore resource_1;  
semaphore resource_2;
```

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

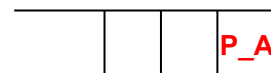
resource_1

-1



resource_2

-1



CONCETTI INTRODUTTIVI

Cap 6: par. 6.2, 6.3

Definizione di deadlock

- *Un set di processi è in **stato di deadlock** se ciascun processo del set è in attesa di un evento che può essere provocato solo da un altro processo del set.*
 - caso "tipico": *deadlock delle **risorse*** (focus della lezione, ma non l'unica possibilità):
 - risorse **hardware** (per es., una stampante) oppure **software** (per es., un record di un databare);
 - altro tipo di deadlock: *deadlock delle **comunicazioni**.*

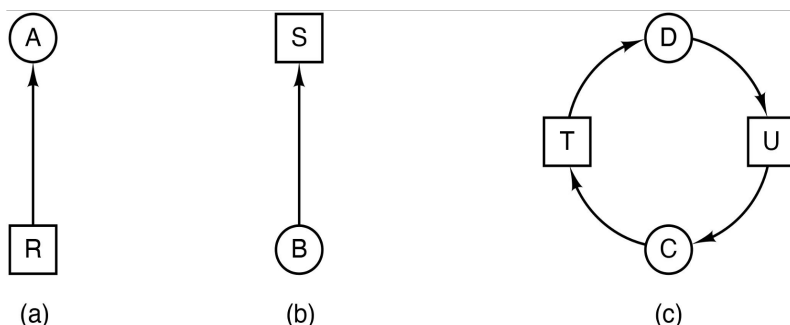
Condizioni per i deadlock delle risorse

- Un deadlock può avvenire se sussistono 4 condizioni:
 - *mutua esclusione* (ciascuna risorsa è assegnata a 1 processo oppure è libera);
 - *possesso e attesa* (i processi che detengono risorse possono chiederne di nuove);
 - *impossibilità di prelazione** (le risorse non possono essere "espropriate" a un processo, ma devono essere rilasciate dal processo che le detiene);
 - *attesa circolare*.

*NOTA: in generale, una risorsa può essere *prelazionabile* (ossia può essere *tolta* al processo che la possiede) o *non prelazionabile*.

Modellazione (resource allocation graph)

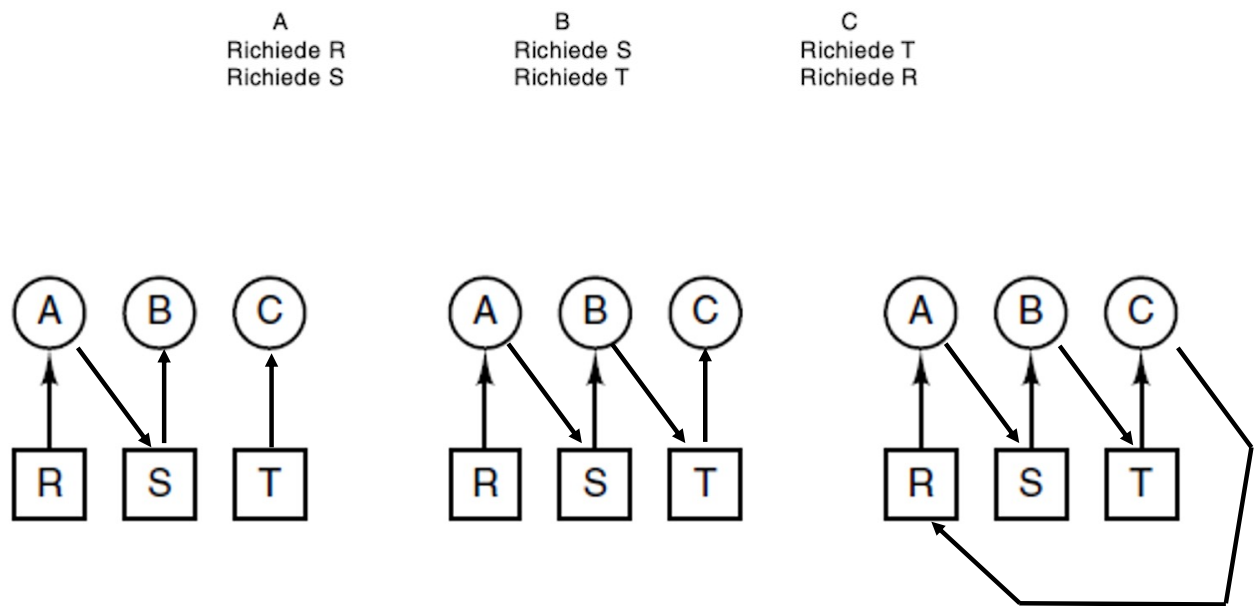
- Processi (*cerchi*), risorse (*quadrati*):
 - $R \rightarrow A$ (fig. a): la risorsa R (*che è stata, in precedenza, richiesta e assegnata ad A*) è attualmente posseduta dal processo A;
 - $B \rightarrow S$ (fig. b): il processo B sta "aspettando" la risorsa S.



C è in attesa di T
(con T posseduta da D)

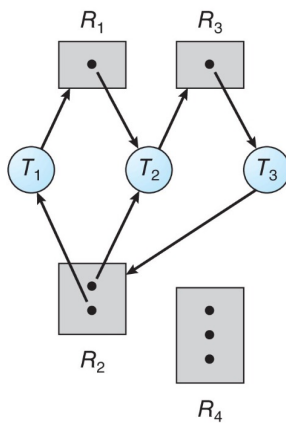
D è in attesa di U
(con U posseduta da C)

Un esempio più complesso

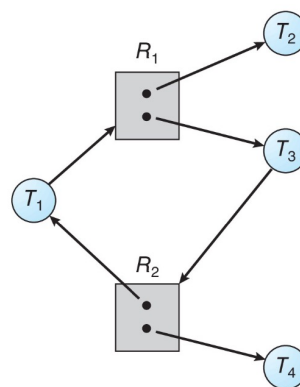


Deadlock

- **Due scenari:** una risorsa per tipo; più risorse per tipo.



ciclo **con** deadlock



ciclo **senza** deadlock

Deadlock

- Se un grafo **non ha** cicli → **no** deadlock;
- Se un grafo **ha** cicli:
 - nel caso di *1 risorsa* per tipo → deadlock;
 - nel caso di *più risorse* per tipo → **possibilità** di deadlock.

Gestione-risoluzione dei deadlock

- È possibile ricorrere a **quattro strategie**:
 - **ignorare** i deadlock;
 - **rilevamento** (e **risoluzione**) dei deadlock (*detection-recovery*);
 - **evitare** i deadlock (*deadlock avoidance*);
 - **prevenzione** dei deadlock (*deadlock prevention*):
 - **prevention**: ossia fare in modo che non si verifichino, *strutturalmente*, una delle 4 condizioni per i deadlock delle risorse.

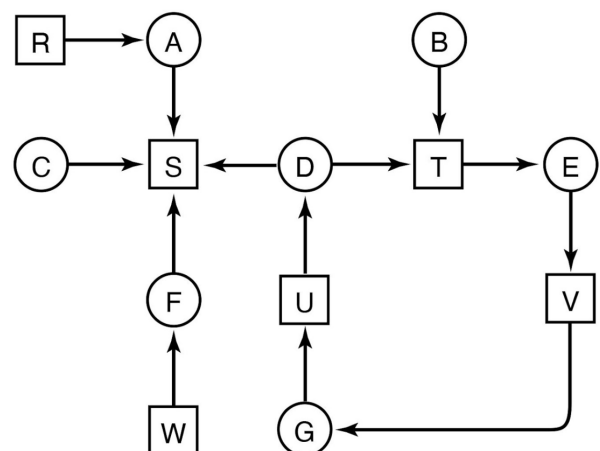
DEADLOCK DETECTION E RECOVERY

Cap 6: par. 6.4 (escluso algoritmo per il rilevamento dei cicli)

Primo caso: 1 risorsa per tipo

1. Il processo *A* detiene *R* e vuole *S*.
2. Il processo *B* non detiene niente ma vuole *T*.
3. Il processo *C* non detiene niente ma vuole *S*.
4. Il processo *D* detiene *U* e vuole *S* e *T*.
5. Il processo *E* detiene *T* e vuole *V*.
6. Il processo *F* detiene *W* e vuole *S*.
7. Il processo *G* detiene *V* e vuole *U*.

se il grafo contiene uno (o più) cicli,
esiste un deadlock.



Secondo caso: **più** risorse per tipo

risorse
esistenti

E = (4 2 3 1)

risorse
disponibili
$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

$$\sum_{i=1}^n C_{ij} + A_j = E_j$$

**matrice di
allocazione corrente**

$C_{i,j}$ = istanze della risorsa j possedute dal processo i

Secondo caso: più risorse per tipo

risorse
esistenti
$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$
risorse
disponibili
$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Request matrix

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

**matrice delle
richieste**

**matrice di
allocazione corrente**

$C_{i,j}$ = istanze della risorsa j possedute dal processo i

$R_{i,j}$ = istanze della risorsa j richieste dal processo i

Secondo caso: più risorse per tipo

Algoritmo di **deadlock detection**:

inizializzazione: tutti i processi sono indicati come "non contrassegnati".

- 1) cerca un processo "non contrassegnato" i per cui la i -esima riga di R sia $\leq A$;
- 2) se tale processo **esiste**, aggiungi la i -esima riga di C ad A , **contrassegna il processo**, e torna a 1;
- 3) se tale processo **non esiste**, l'algoritmo **termina**.

tutti i processi che rimangono "non contrassegnati" sono in deadlock

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Tape drives Plotters Scanners Cameras

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Request matrix

$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$

Tape drives Plotters Scanners Cameras

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Current allocation matrix

Secondo caso: più risorse per tipo

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

$$A + C_{2,j}$$

$$A = \begin{pmatrix} 2 & 2 & 2 & 0 \end{pmatrix}$$

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Tape drives Plotters Scanners Cameras

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Request matrix

P_0
 P_1
 P_2

$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$

Tape drives Plotters Scanners Cameras

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Current allocation matrix

P_0
 P_1
 P_2

Secondo caso: **più** risorse per tipo

$$A = (\quad 2 \quad \quad 1 \quad \quad 0 \quad \quad 0 \quad)$$

// la riga R_2 (processo P_2) è $\leq A \dots$

$$A + C_{2,j}$$

$$A = (\quad 2 \quad \quad 2 \quad \quad 2 \quad \quad 0 \quad)$$

// la riga R_1 (processo P_1) è $\leq A \dots$

$$A + C_{1,j}$$

$$A = (\quad 4 \quad \quad 2 \quad \quad 2 \quad \quad 1 \quad)$$

// la riga R_0 (processo P_0) è $\leq A \dots$

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 \end{pmatrix}$$

Tape drives
 Plotters
 Scanners
 Cameras

Request matrix

$$\begin{matrix} P_0 \\ P_1 \\ P_2 \end{matrix} R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

$$E = (\quad 4 \quad \quad 2 \quad \quad 3 \quad \quad 1 \quad)$$

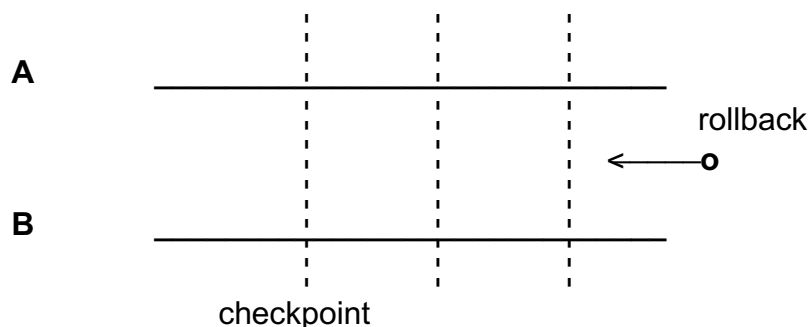
Tape drives
 Plotters
 Scanners
 Cameras

Current allocation matrix

$$\begin{matrix} P_0 \\ P_1 \\ P_2 \end{matrix} C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Risoluzione del deadlock

- Possibili approcci per risolvere il deadlock:
 - prelazione della risorsa;
 - eliminazione di processi;
 - checkpoint & rollback.



DEADLOCK AVOIDANCE

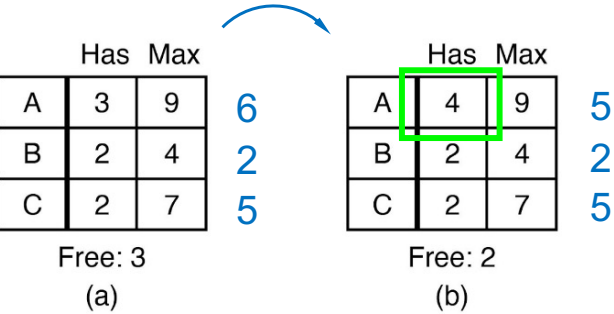
Cap 6: par. 6.5 (escluso 6.5.1)

Evitare i deadlock

- Intuizione: *evitare* i deadlock **decidendo** se l'assegnazione di una risorsa sia "**sicura**" oppure no;
- Richiede delle informazioni *in anticipo*:
 - ciascun processo *dichiara* il **massimo numero** di risorse di ciascun tipo di cui ha bisogno.

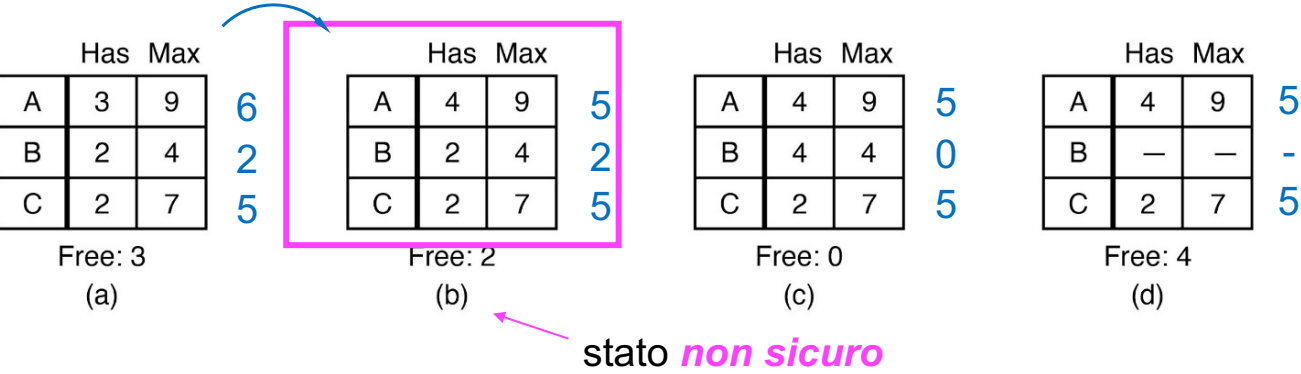
Stati sicuri e non sicuri

A chiede 1 risorsa...



Stati sicuri e non sicuri

A chiede 1 risorsa...



Stati sicuri e non sicuri

- Uno stato è detto **sicuro** se esiste un ordine di scheduling in cui *ogni processo* può essere eseguito sino alla conclusione anche se tutti i processi richiedono, all'improvviso, il loro numero massimo di risorse.

	Ha	Max
A	3	9
B	2	4
C	2	7

Libere: 3

(a)

*non concedo 1 ad A,
ma, piuttosto, vengono assegnate
le 2 risorse a B*

Stati sicuri e non sicuri

- Uno stato è detto **sicuro** se esiste un ordine di scheduling in cui *ogni processo* può essere eseguito sino alla conclusione anche se tutti i processi richiedono, all'improvviso, il loro numero massimo di risorse.

	Ha	Max
A	3	9
B	2	4
C	2	7

Libere: 3
(a)

	Ha	Max
A	3	9
B	4	4
C	2	7

Libere: 1
(b)

	Ha	Max
A	3	9
B	0	–
C	2	7

Libere: 5
(c)

	Ha	Max
A	3	9
B	0	–
C	7	7

Libere: 0
(d)

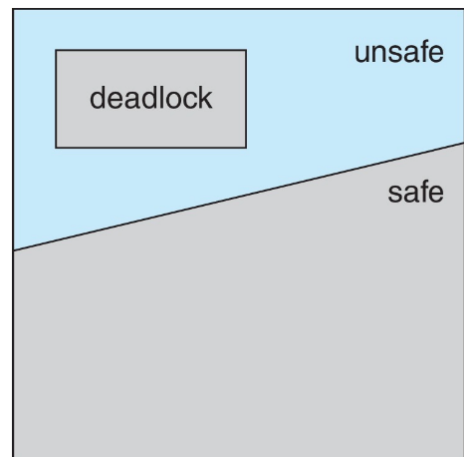
	Ha	Max
A	3	9
B	0	–
C	0	–

Libere: 7
(e)

stato sicuro

Stati sicuri e non sicuri

- Se il sistema è in uno stato **sicuro** → no deadlock;
- Se il sistema è in uno stato **non sicuro** → possibilità di deadlock;
- Avoidance**: assicurare che il sistema **non entri** mai in uno stato *non sicuro*.



Algoritmo del banchiere (un tipo di risorsa)

- L'algoritmo controlla se, *acconsentire* ad una richiesta, conduca ancora ad uno stato **sicuro**:
 - se sì, la richiesta è concessa; altrimenti, è rinviata;

stato sicuro

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max	
A	1	6	5
B	1	5	4
C	2	4	2
D	4	7	3

Free: 2

(b)

Algoritmo del banchiere (un tipo di risorsa)

- L'algoritmo controlla se, *acconsentire* ad una richiesta, conduca ancora ad uno stato **sicuro**:
 - se sì, la richiesta è concessa; altrimenti, è rinviata;

stato sicuro

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)



B chiede 1 risorsa...

Algoritmo del banchiere (un tipo di risorsa)

- L'algoritmo controlla se, *acconsentire* ad una richiesta, conduca ancora ad uno stato **sicuro**:
 - se sì, la richiesta è concessa; altrimenti, è rinviata;

stato sicuro

stato non sicuro

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)



	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

Algoritmo del banchiere (più tipi di risorsa)

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	

Risorse assegnate

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	

Risorse ancora necessarie*

E = (6342) //esistenti
P = (5322) //possedute
A = (1020) //disponibili

*I processi devono dichiarare le loro necessità di **risorse totali** prima dell'esecuzione in modo che il sistema possa, in ogni momento, calcolare le "risorse ancora necessarie"

Algoritmo del banchiere (più tipi di risorsa)

- Algoritmo per verificare **se uno stato è sicuro**:
 - 1)individua un processo la cui riga di $R \leq A$. Se non esiste una tale riga-processo → deadlock;
 - 2)se tale processo **esiste**, aggiungi le sue risorse ad A e **contrassegna il processo** come "terminato";
 - 3)ripeti 1-2 finché i processi tutti i processi risultano in "terminato" (lo stato è **sicuro**) oppure non c'è alcun processo la cui richiesta può essere soddisfatta (lo stato è **non sicuro**).

Algoritmo del banchiere (più tipi di risorsa)

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	

Risorse assegnate

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	

Risorse ancora necessarie

$E = (6342)$ //esistenti
 $P = (5322)$ //possedute
 $A = (1020)$ //disponibili
 $A = (2121)$

Algoritmo del banchiere (più tipi di risorsa)

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	3	0	1	1	
B	0	1	0	0	
C	1	1	1	0	
D	1	1	0	1	
E	0	0	0	0	

Risorse assegnate

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
A	1	1	0	0	
B	0	1	1	2	
C	3	1	0	0	
D	0	0	1	0	
E	2	1	1	0	

Risorse ancora necessarie

$E = (6342)$ //esistenti
 $P = (5322)$ //possedute
 $A = (1020)$ //disponibili
 $A = (2121)$

T

Algoritmo del banchiere (più tipi di risorsa)

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
T	A	3	0	1	1
	B	0	1	0	0
	C	1	1	1	0
	D	1	1	0	1
	E	0	0	0	0
	Risorse assegnate				
	A	1	1	0	0
	B	0	1	1	2
	C	3	1	0	0
	D	0	0	1	0
	E	2	1	1	0
	Risorse ancora necessarie				

$E = (6342)$ //esistenti
 $P = (5322)$ //possedute
 $A = (1020)$ //disponibili
 ~~$A = (2121)$~~
 $A = (5132)$

Algoritmo del banchiere (più tipi di risorsa)

	Processo	Unità a nastro	Plotter	Stampanti	Telecamere
T	A	3	0	1	1
	B	0	1	0	0
	C	1	1	1	0
T	D	1	1	0	1
	E	0	0	0	0
	Risorse assegnate				
	A	1	1	0	0
	B	0	1	1	2
	C	3	1	0	0
	D	0	0	1	0
	E	2	1	1	0
	Risorse ancora necessarie				

$E = (6342)$ //esistenti
 $P = (5322)$ //possedute
 $A = (1020)$ //disponibili
 ~~$A = (2121)$~~
 $A = (5132)$
 ...
lo stato è sicuro

CONSIDERAZIONI FINALI

Cap 6: par. 6.7.3, 6.7.4.

Confronto approcci

- ❑ Deadlock avoidance: difficile conoscere il bisogno di risorse *a priori*;
- ❑ Deadlock prevention: usato in applicazioni specifiche;
- ❑ Deadlock detection: possibile, ma necessari appropriati meccanismi di detection-recovery;
- ❑ Ultima spiaggia: **ignorare** il problema.

Livelock

- Assenza di progresso dell'elaborazione, sebbene i processi **non** siano bloccati.

```
void process_A(void) {  
    acquire_lock(&resource_1);  
    while (try_lock(&resource_2) == FAIL) {  
        release_lock(&resource_1);  
        wait_fixed_time();  
        acquire_lock(&resource_1);  
    }  
    use_both_resources( );  
    release_lock(&resource_2);  
    release_lock(&resource_1);  
}
```

```
void process_B(void) {  
    acquire_lock(&resource_2);  
    while (try_lock(&resource_1) == FAIL) {  
        release_lock(&resource_2);  
        wait_fixed_time();  
        acquire_lock(&resource_2);  
    }  
    use_both_resources();  
    release_lock(&resource_1);  
    release_lock(&resource_2);  
}
```

Starvation

- Attesa indefinita di una risorsa:
 - per es., scheduling basato su priorità o shortest-remaining-time-next;
- Differente** dal deadlock: il processo in deadlock è bloccato; il processo che rischia la *starvation* potrebbe provare a fare altro.

